same method as Kernel#load, and it overrides the load method in Loadable. (If that sounds confusing, try drawing a picture of MyClass's ancestors chain, and everything will be clear.)

This trick is an example of the power of UnboundMethods, but it's also a contrived solution to a very specific problem—a solution that leaves you with a confusing chain of ancestors that contains three load methods, two of which are identical to each other (Kernel#load and MyClass#load), and two of which are never called (Kernel#load and Loadable#load). It's probably good policy not to try this kind of class hacking at home.

Callable Objects Wrap-Up

Callable objects are snippets of code that you can evaluate, and they carry their own scope along with them. They can be the following:

- *Blocks* (they aren't really "objects," but they are still "callable"): Evaluated in the scope in which they're defined.
- *Procs*: Objects of class Proc. Like blocks, they are evaluated in the scope where they're defined.
- *Lambdas*: Also objects of class Proc but subtly different from regular procs. They're closures like blocks and procs, and as such they're evaluated in the scope where they're defined.
- *Methods*: Bound to an object, they are evaluated in that object's scope. They can also be unbound from their scope and rebound to another object or class.

Different callable objects exhibit subtly different behaviors. In methods and lambdas, return returns from the callable object, while in procs and blocks, return returns from the callable object's original context. Different callable objects also react differently to calls with the wrong arity. Methods are stricter, lambdas are almost as strict (save for some corner cases), and procs and blocks are more tolerant.

These differences notwithstanding, you can still convert from one callable object to another, such as by using Proc.new, Method#to_proc, or the & operator.

Writing a Domain-Specific Language

Where you and Bill, at long last, write some code.

"Enough talking about blocks," Bill says. "It's time to focus on today's job. Let's call it the RedFlag project."